

Pytorch 입문 부트캠프

3. Pytorch Exercises (3) Vision Classification

목차

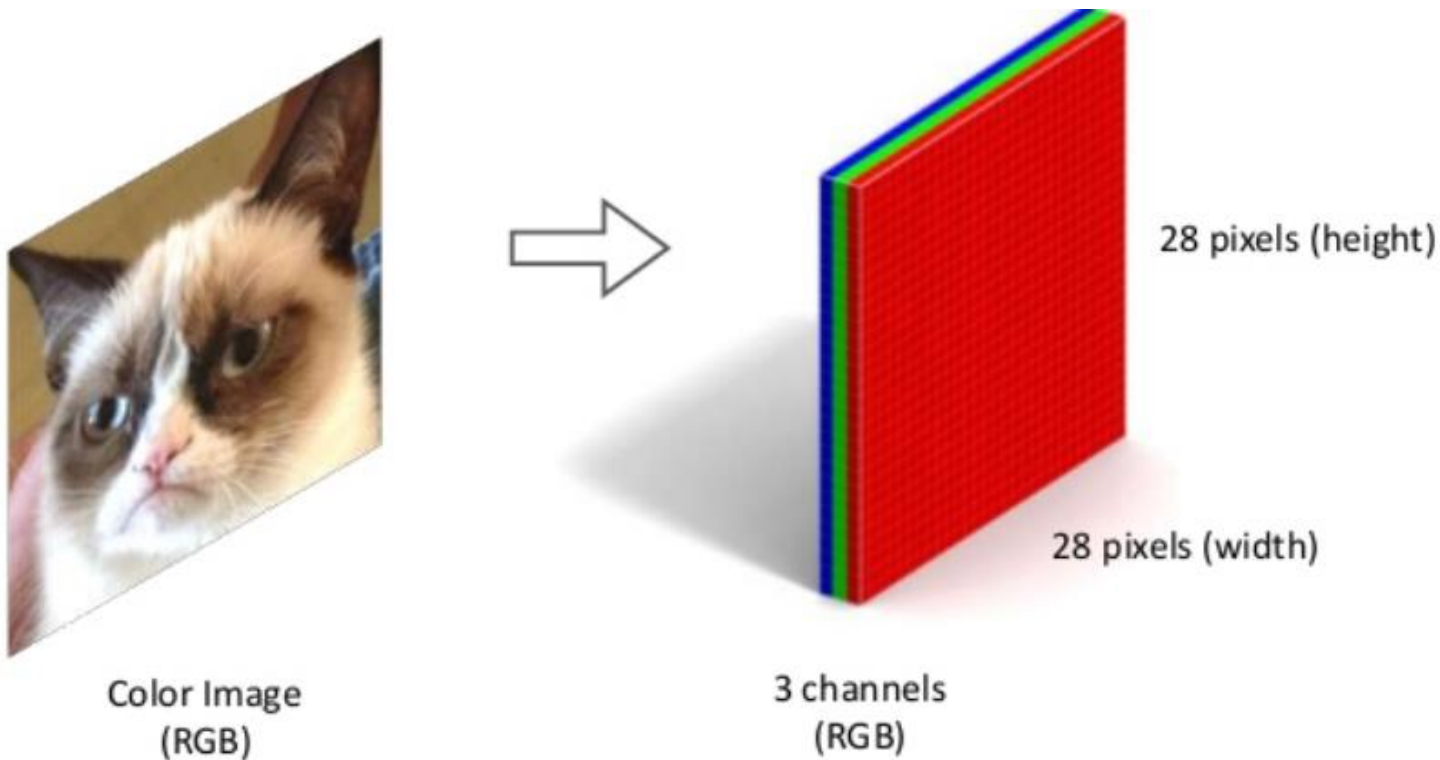
- 1 Vision Classification
- 2 Convolutional Neural Network
- 3 CNN in Pytorch

1

Vision Classification

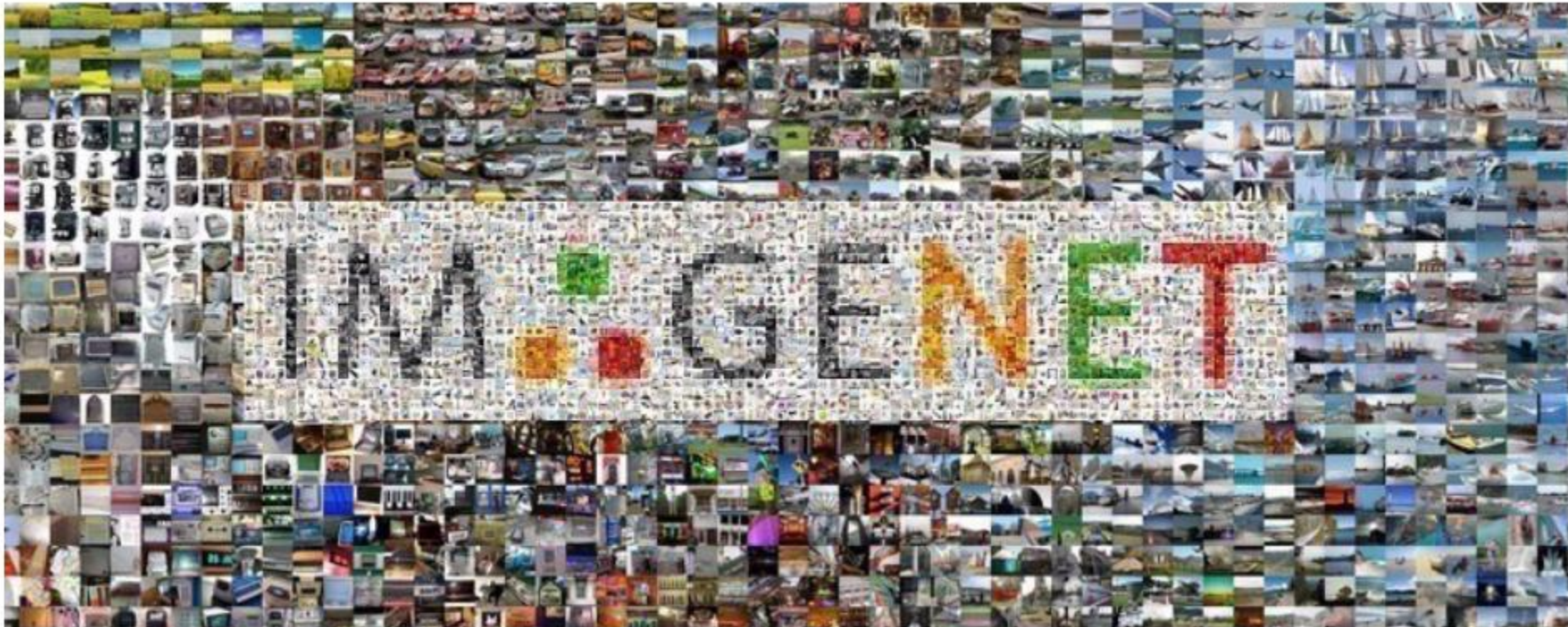
Vision Classification

- Image Data
 - 대표적인 비정형 데이터
 - 사람은 이미지를 눈으로 보고 바로 인식할 수 있지만, 컴퓨터는 이미지를 각 픽셀에 입력되어 있는 숫자 값들의 배열로 인식 → 결국 Tensor!
 - 컬러 이미지는 RGB의 3개 채널로 구성, 흑백은 1개 채널로 구성



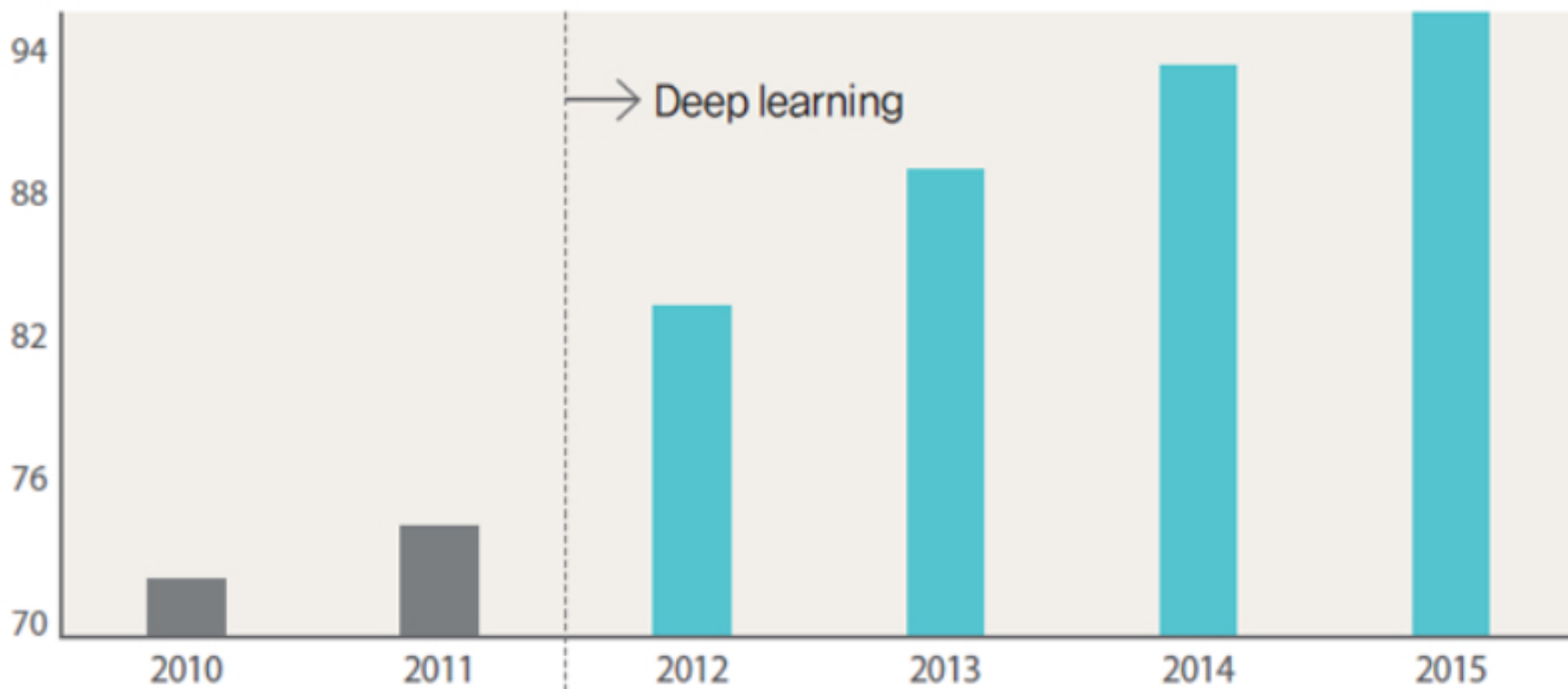
Vision Classification

- Vision Classification
 - 인간의 시각적 인지능력을 따라잡기 위한 연구 분야인 Computer Vision의 대표적인 Task
 - ImageNet Contest 2012(ILSVRC*)에 Alexnet이 최초로 Convolutional Neural Network 구조를 사용
 - 당시 2위 알고리즘과 10%이상의 성능차이를 보이며 우승, 이후 Vision Transformer 전까지 CNN이 Vision 분야의 가장 대표적인 모델로 등극



Vision Classification

- Convolutional Neural Network?
 - Convolutional Layer와 Pooling Layer를 포함하는 신경망 구조
 - 이미지 처리에 좋은 성능을 보임
 - Alexnet, VGG, Resnet, Google Inception Net 등



Vision Classification

- 사용할 Dataset : Fashion MNIST
 - 28 x 28 크기의 Grey scale 이미지로, 각종 의류에 대한 이미지 데이터
 - 총 60000개의 학습셋과 10000개의 테스트 셋

0 : T-shirt/top

T_shirt



1 : Trouser

Trouser



2 : Pullover

Pullover



3 : Dress

Dress



4 : Coat

Coat



5 : Sandal

Sandal



6 : Shirt

Shirt



7 : Sneaker

Sneaker



8 : Bag

Bag



9 : Ankel boot

Ankle boot

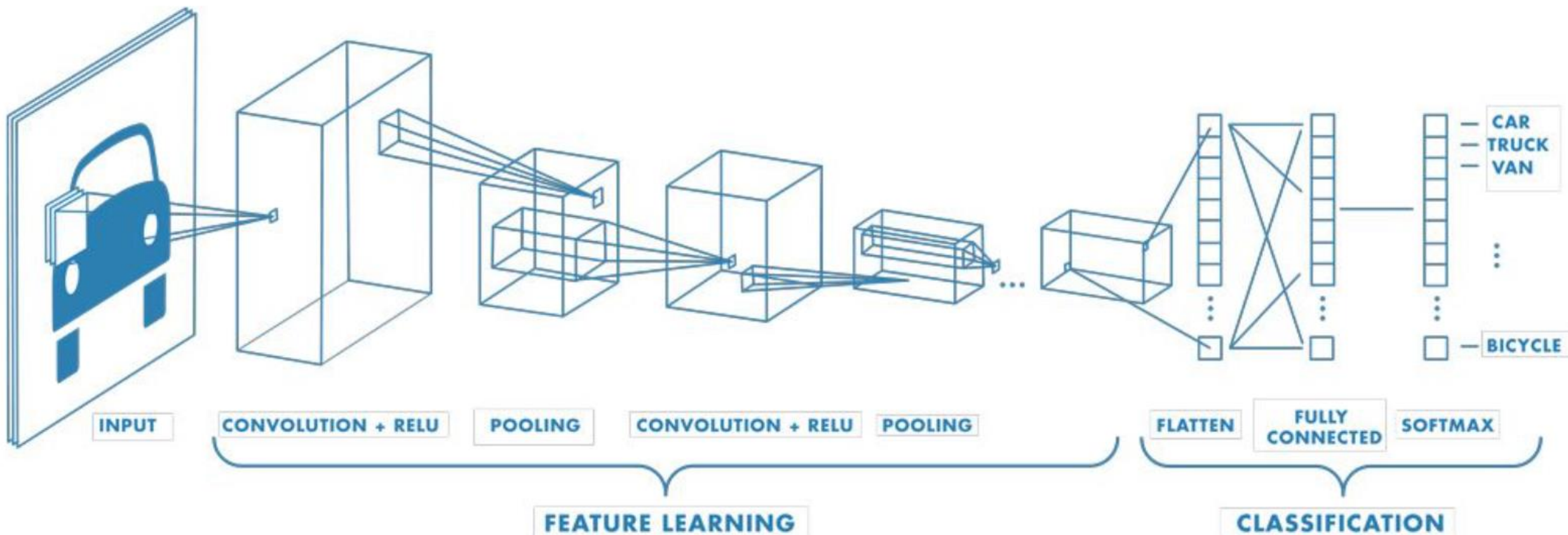


2

Convolutional Neural Network

Convolutional Neural Network

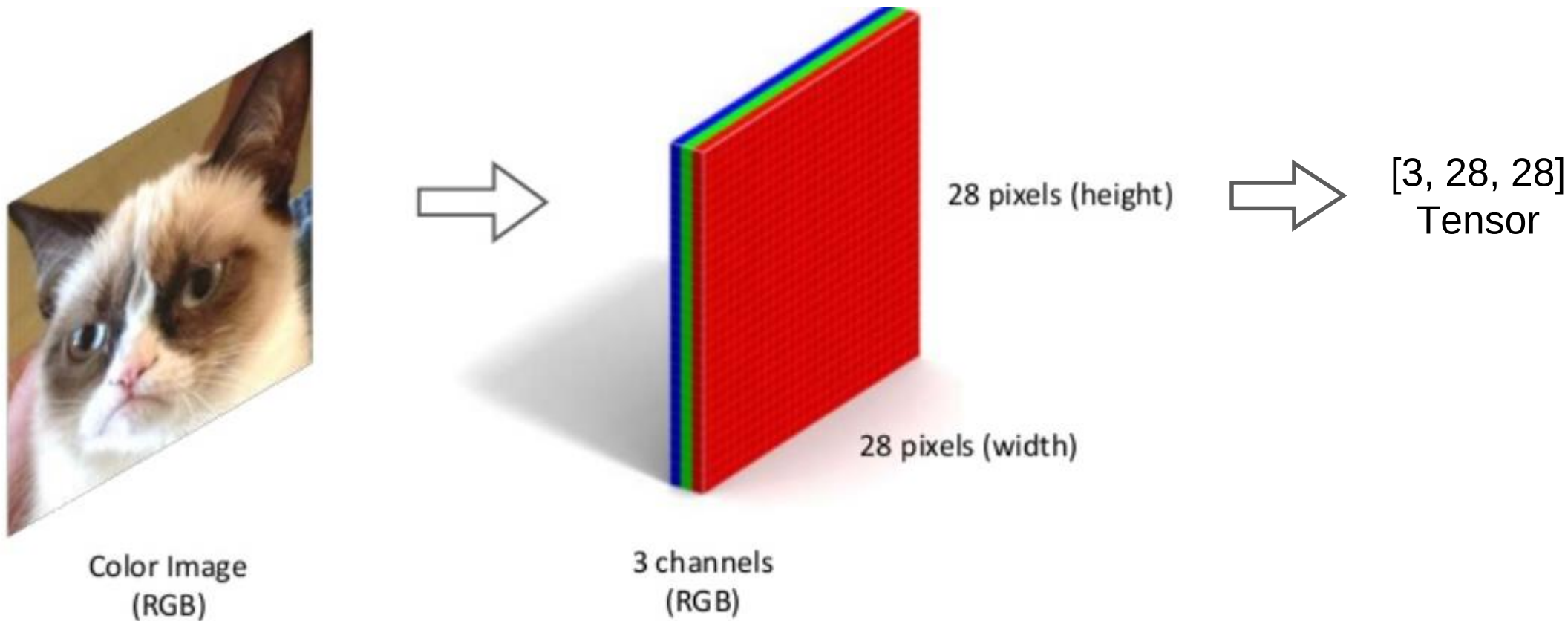
- Convolutional Neural Networks
 - CNN, ConvNet, 합성곱 신경망
 - Convolutional Layer + Pooling Layer로 이미지에서 정보를 추출한 뒤(feature extractor)
추출된 feature (=representation) 을 FC layer로 분류 (=Classifier)



Convolutional Neural Network

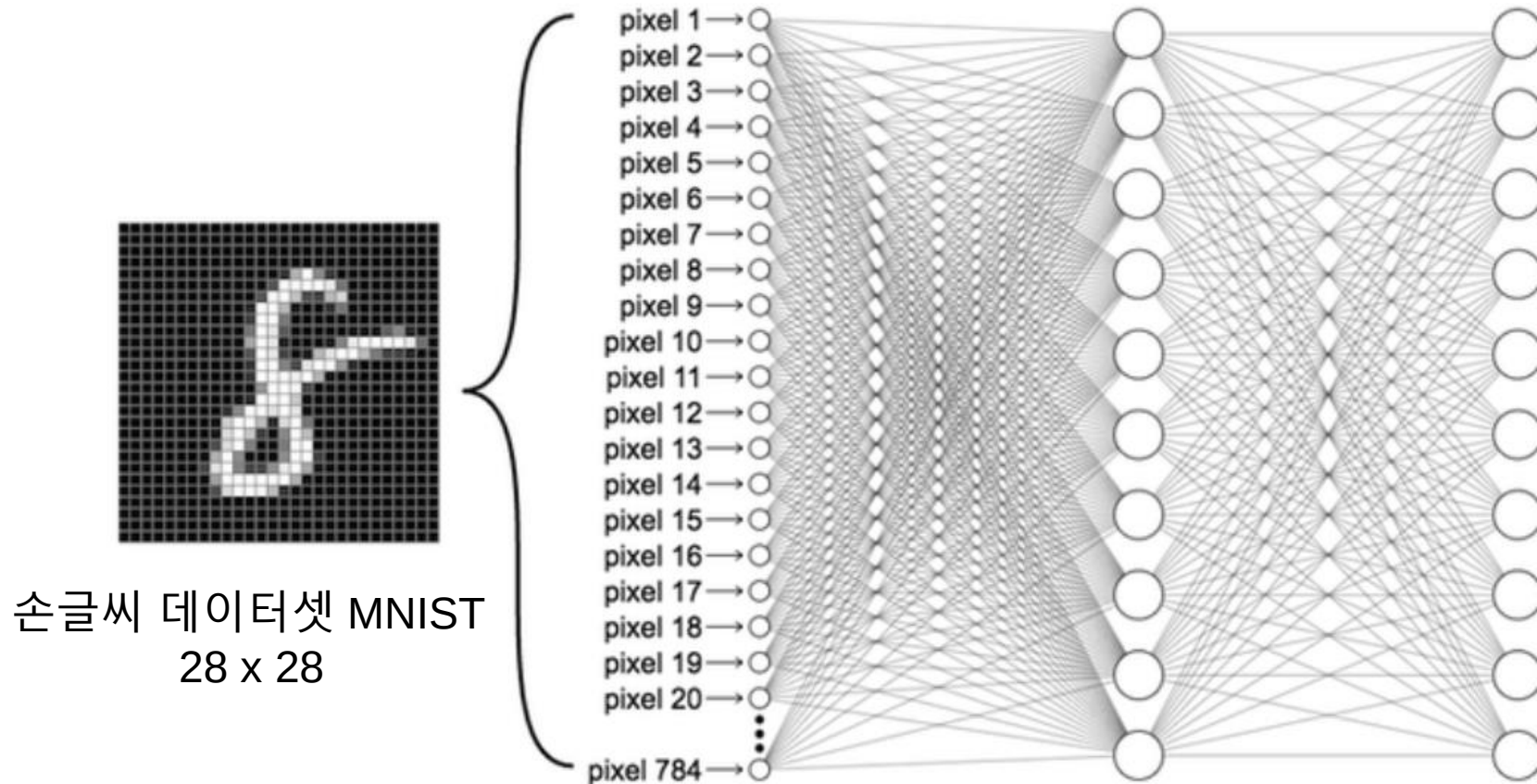
- Image Data

- 사람은 이미지를 눈으로 보고 바로 인식할 수 있지만, 컴퓨터는 이미지를 각 픽셀에 입력되어 있는 숫자 값들의 배열로 인식
- 컬러 이미지는 RGB의 3개 채널로 구성, 흑백은 1개 채널로 구성



Convolutional Neural Network

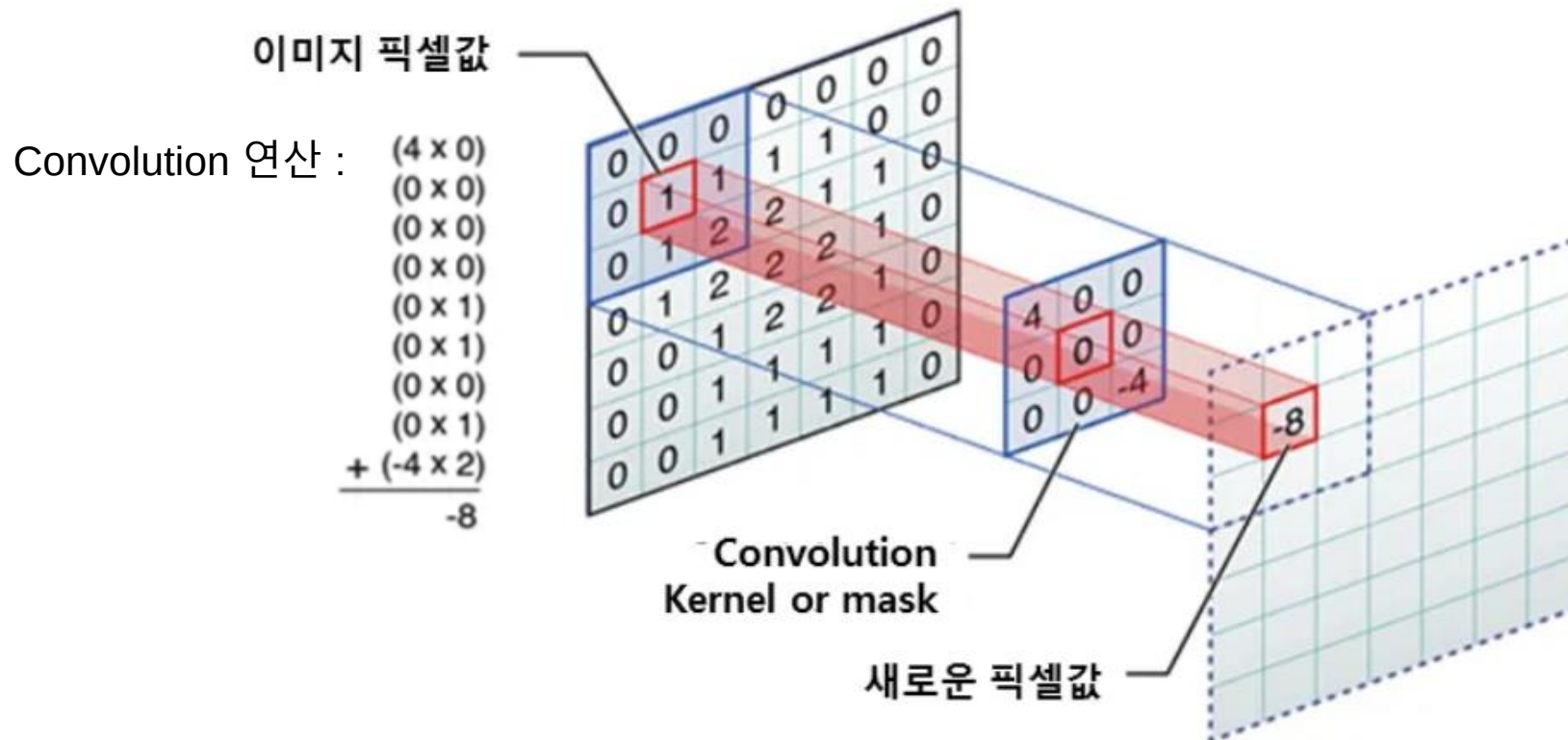
- FC Layer로 Image Data는 처리할 수 없나?
 - Image Data를 모두 flatten 시켜서 FC Layer로 학습 시키게 되면 필요한 parameter의 수가 너무 많아짐
→ 모델 학습의 비효율성
 - 이미지의 특정 패턴을 학습하기 어려움



Convolutional Neural Network

- Convolutional Operation

- Convolution Filter(=Kernel)이 이동하면서 픽셀과 합성곱을 통해 Feature Map을 생성
- Convolution Filter의 값은 학습의 대상(parameter)으로, 점차 각 filter마다 이미지의 특정 패턴(선, 색상, 질감 등)을 학습하게 됨
- Filter size는 주로 3x3, 5x5 등 정사각형 모양을 주로 사용함



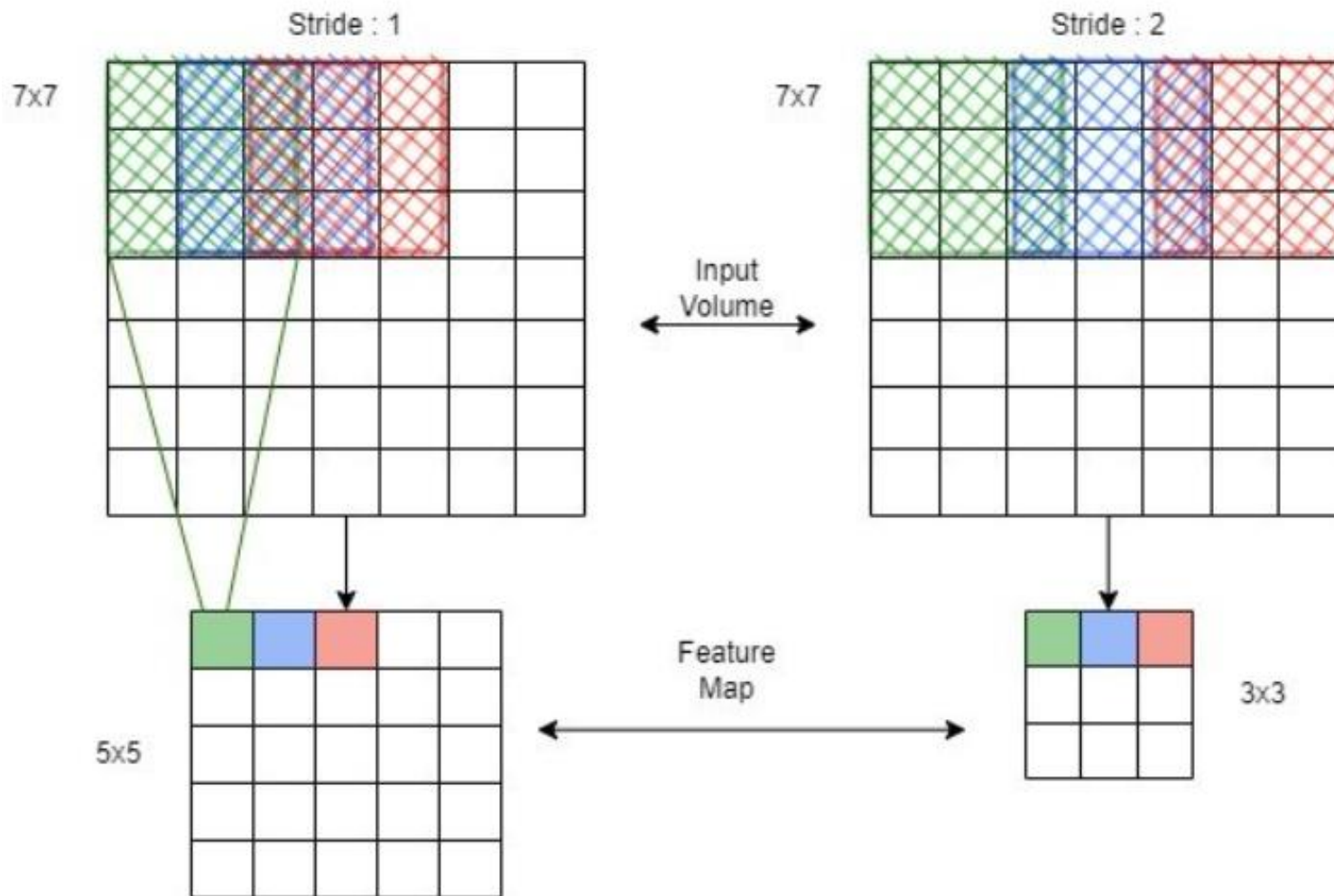
Convolutional Neural Network

- Convolutional Operation with Multi channel
 - RGB 이미지인 3 channel 의 3D Tensor를 대상으로 convolutional operation

<u>Input</u>	<u>Kernel</u>	<u>Intermediate Output</u>	
1 0 1 0 2 1 1 3 2 1 1 1 0 1 1 2 3 2 1 3 0 2 0 1 0	0 1 0 0 0 2 0 1 0	7 5 3 4 7 5 7 2 8	
1 0 0 1 0 2 0 1 2 0 3 1 1 3 0 0 3 0 3 2 1 0 3 2 1	2 1 0 0 0 0 0 3 0	5 3 10 13 1 13 7 12 11	<u>Output</u> 19 13 15 28 16 20 23 18 25
2 0 1 2 1 3 3 1 3 2 2 1 1 1 0 3 1 3 2 0 1 1 2 1 1	1 0 0 1 0 0 0 0 2	7 5 2 11 8 2 9 4 6	

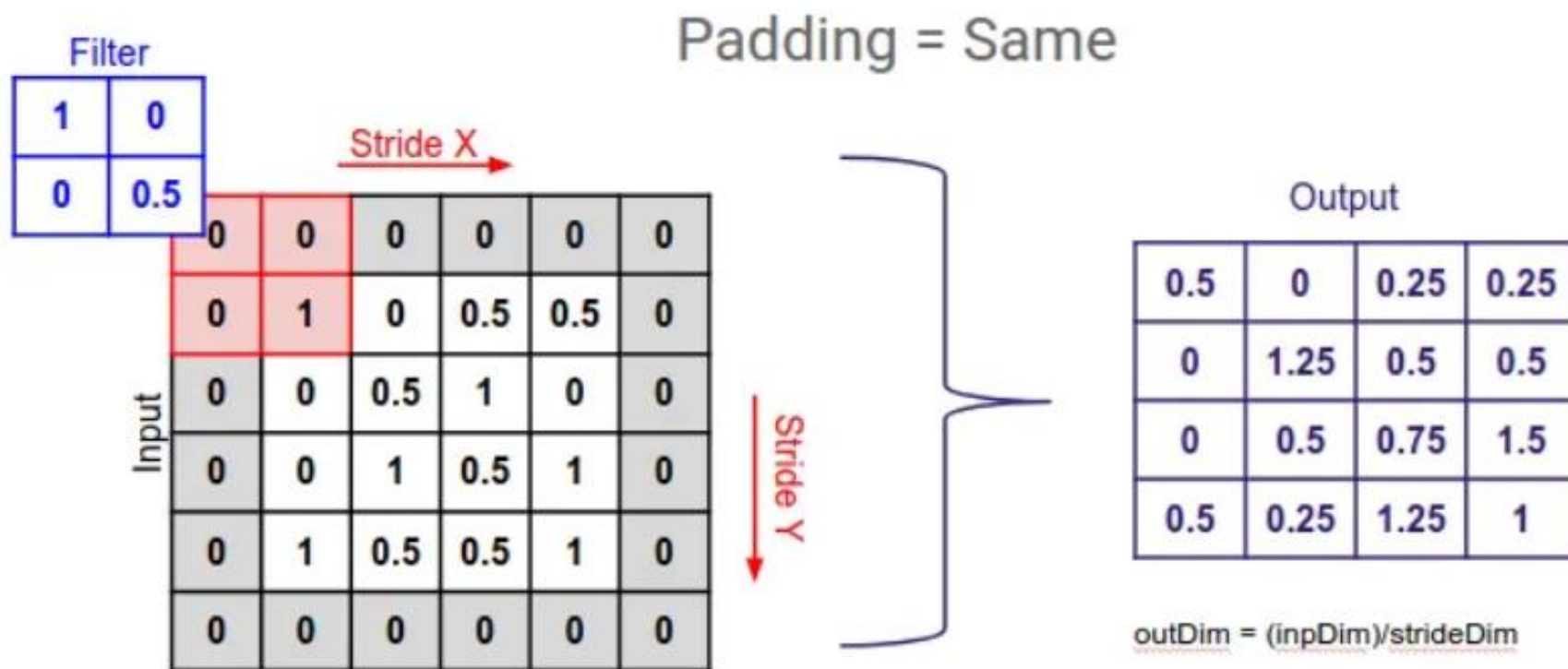
Convolutional Neural Network

- Hyperparameters in Convolutional Layer
 - Stride : Conv filter 적용 시 이동 칸 수, stride를 늘릴수록 feature map은 작아짐



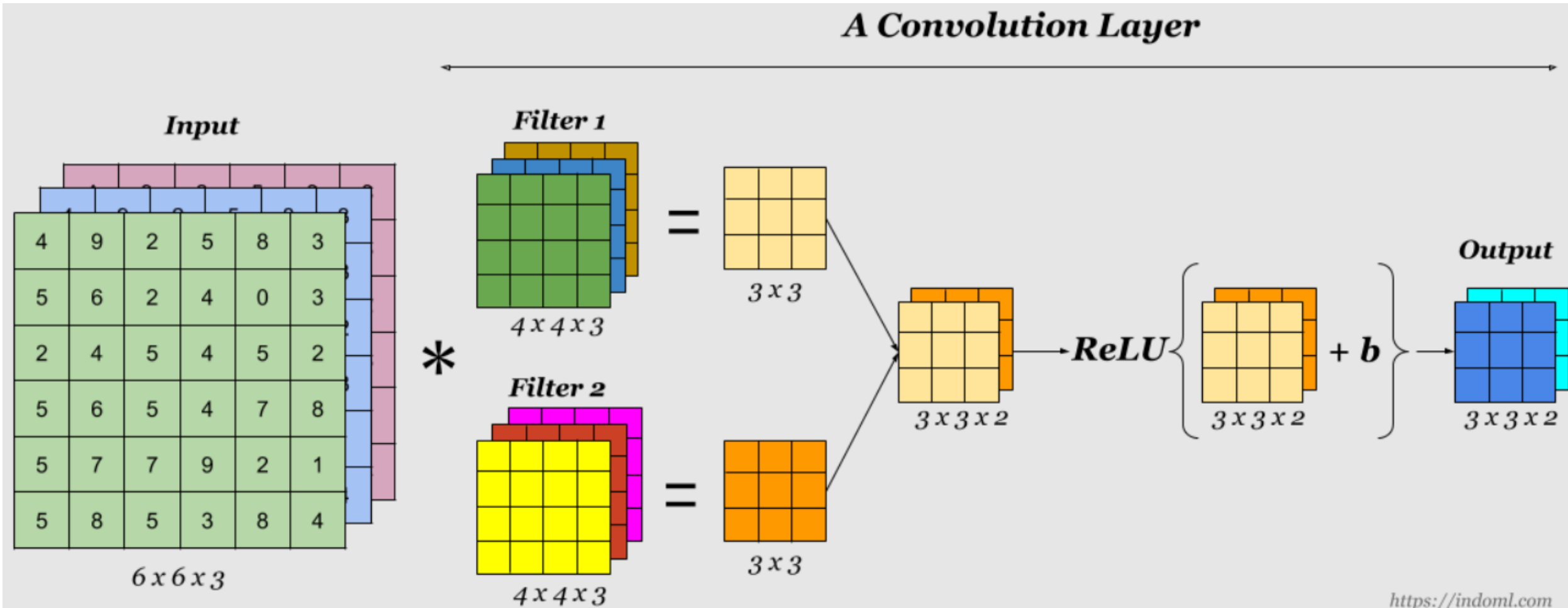
Convolutional Neural Network

- Hyperparameters in Convolutional Layer
 - Padding : Filter를 통과한 feature map의 크기는 입력 데이터보다 작아지기 때문에 이를 방지하기 위해 가장자리에 0을 채워 넣는 것



Convolutional Neural Network

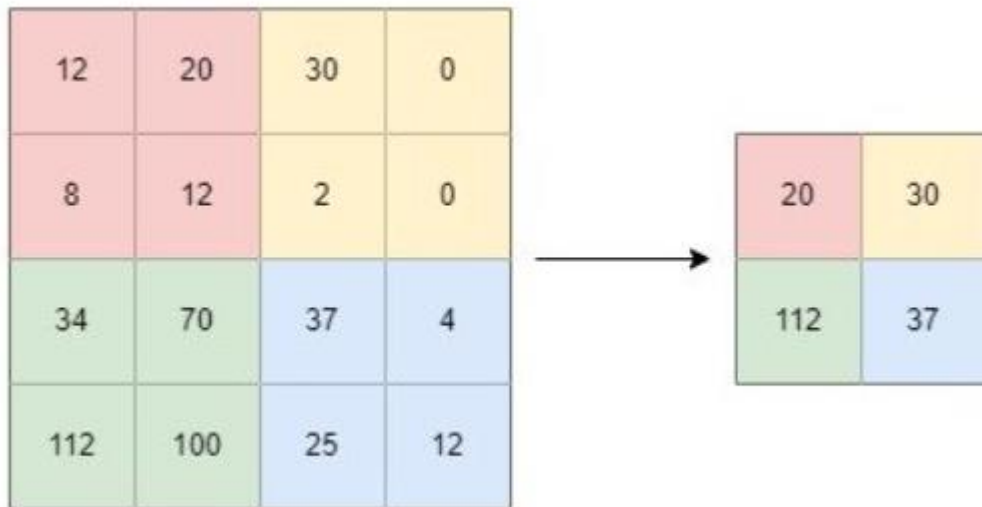
- Single Conv Layer
 - Stride = 1, Padding = 0 을 가정했을 때,



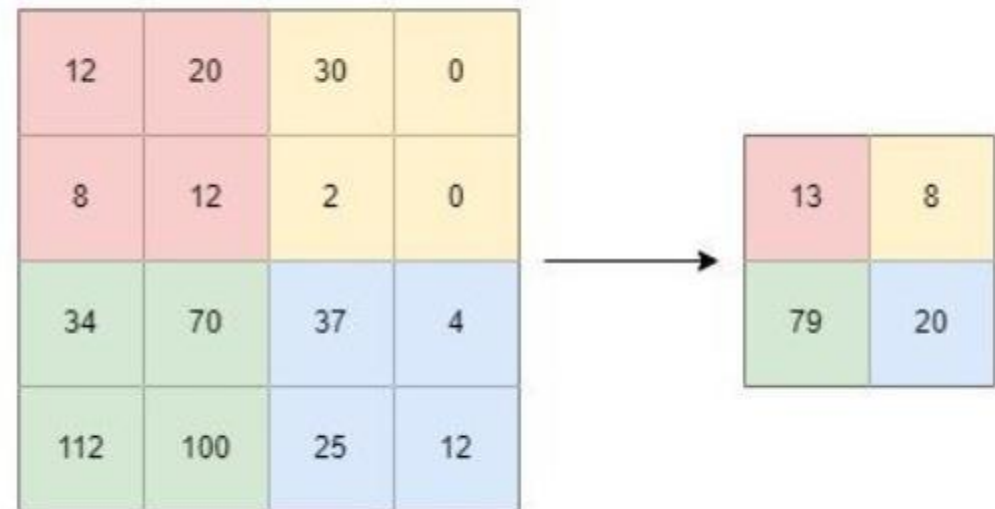
Convolutional Neural Network

- Pooling Layer
 - Dimension의 크기를 줄이고 싶을 때 사용하는 down sampling 방식
 - 주로 CNN은 Pooling을 이용하여 점점 feature map size는 작아지고, Depth는 깊어지는 구조로 구성되어 있음

[Max Pooling]



[Average Pooling]



3

CNN in Pytorch

CNN in Pytorch

- Conv Layer

- Input Tensor shape

```
# Define the input tensor  
input = torch.randn(10, 3, 7, 7) # Batch_size, Kernel_size, Height, Weight
```

- Conv Layer 정의

```
# Define the convolutional layer  
conv = nn.Conv2d(in_channels=3, out_channels=2, kernel_size=3, stride=1, padding='same')
```

- Output shape

```
# Apply the convolution  
output = conv(input)  
  
# Print the output shape  
print(output.shape)  
  
torch.Size([10, 2, 7, 7])
```

CNN in Pytorch

- 데이터셋 구성

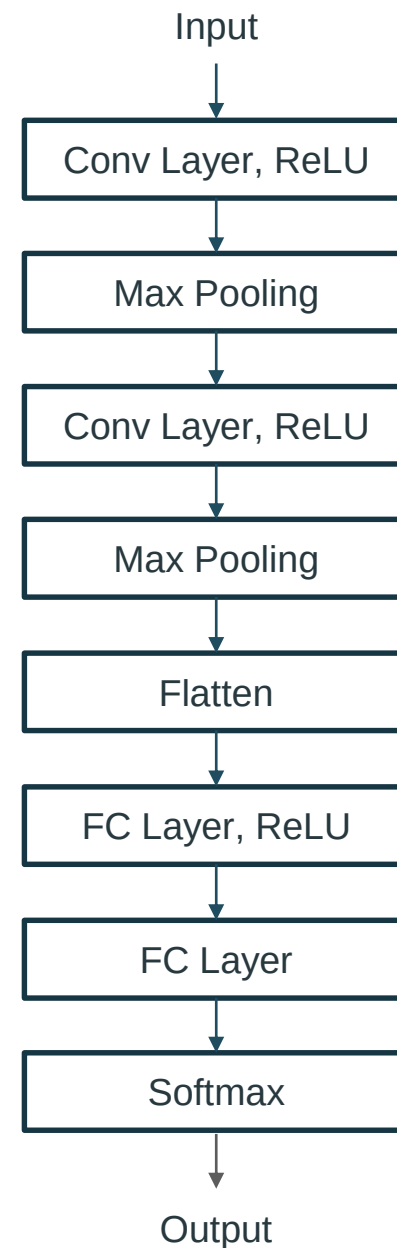
```
train_loader = torch.utils.data.DataLoader(  
    datasets.FashionMNIST("data", train=True, download=True, transform=transforms.Compose([  
        transforms.ToTensor(),  
        transforms.Normalize((0.1307,), (0.3081,))  
    ])),  
    batch_size=64, shuffle=True  
)
```

- datasets : torchvision 아래에 데이터셋 이미 구성되어 있음(FashionMNIST)
 - transforms : 전처리 적용
 - ToTensor : torch 텐서로 취급
 - Normalize : trainset의 평균, 분산으로 픽셀들의 range를 scaling (표준화)

CNN in Pytorch

- CNN 구성

```
class FashionMNISTModel(nn.Module):  
    def __init__(self):  
        super(FashionMNISTModel, self).__init__()  
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3)  
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3)  
        self.fc1 = nn.Linear(64 * 5 * 5, 128)  
        self.fc2 = nn.Linear(128, 10)  
  
    def forward(self, x):  
        x = F.relu(self.conv1(x))  
        x = F.max_pool2d(x, 2)  
        x = F.relu(self.conv2(x))  
        x = F.max_pool2d(x, 2)  
        x = x.view(-1, 64 * 5 * 5)  
        x = F.relu(self.fc1(x))  
        x = self.fc2(x)  
        return F.log_softmax(x, dim=1)  
  
model = FashionMNISTModel()
```



실습 해볼까요?